

Array Creation

```
In [1]: import numpy as np
```

```
In [2]: a = np.array([1,2,3])  
a
```

```
Out[2]: array([1, 2, 3])
```

```
In [3]: print('Array a:',a)
```

```
Array a: [1 2 3]
```

```
In [4]: b = np.arange(0,9,2)  
print('Array a:',b)
```

```
Array a: [0 2 4 6 8]
```

```
In [6]: c = np.zeros((3,4))  
c
```

```
Out[6]: array([[0., 0., 0., 0.],  
              [0., 0., 0., 0.],  
              [0., 0., 0., 0.]])
```

```
In [10]: c = np.zeros((3,4),dtype=int)  
c
```

```
Out[10]: array([[0, 0, 0, 0],  
               [0, 0, 0, 0],  
               [0, 0, 0, 0]])
```

```
In [11]: d = np.ones((3,5))  
d
```

```
Out[11]: array([[1., 1., 1., 1., 1.],  
               [1., 1., 1., 1., 1.],  
               [1., 1., 1., 1., 1.]])
```

```
In [12]: # create an identity matrix  
  
f = np.eye(4)  
print('Identity matrix :\n',f)
```

```
Identity matrix :  
[[1. 0. 0. 0.]  
 [0. 1. 0. 0.]  
 [0. 0. 1. 0.]  
 [0. 0. 0. 1.]
```

Array manipulation function

```
In [13]: # reshape an array  
a1 = np.array([1,2,3,])
```

```
reshape = np.reshape(a1,(3,1))
print('Reshape array :\n',reshape)
```

Reshape array :

```
[[1]
 [2]
 [3]]
```

In [16]: *# flatten an array*

```
a2 =np.array([[1,2,3],[4,5,6],[7,8,9]])
flatten = np.ravel(a2)
print('Flatten array :\n',flatten)
```

Flatten array :

```
[1 2 3 4 5 6 7 8 9]
```

In [18]: *# Transpose array (convert rows to column and vice versa)*

```
a3 =np.array([[1,2,3],[4,5,6],[7,8,9]])
transposed= np.transpose(a3)
print('Transpüose array :\n',transposed)
```

Transpüose array :

```
[[1 4 7]
 [2 5 8]
 [3 6 9]]
```

In [20]: *# stack array vertically*

```
a4 = np.array([11,12,13])
a5 = np.array([21,22,23])
v_stack = np.vstack([a4,a5])
print('vstack array :\n',v_stack)
```

vstack array :

```
[[11 12 13]
 [21 22 23]]
```

In [21]:

```
a6 = np.array([11,12,13])
a7 = np.array([21,22,23])
h_stack = np.hstack([a6,a7])
print('vstack array :\n',h_stack)
```

vstack array :

```
[11 12 13 21 22 23]
```

mathematical function

In [29]: *#add 2 array*

```
m = np.array([11,12,13])
added = np.add(m,2)
print(added)
```

```
[13 14 15]
```

In [24]: *# sqare each element*

```
square = np.power(m,2)
print(square)
```

```
[121 144 169 196]
```

```
In [26]: # square root of an element
square_root = np.sqrt(m)
print(square_root)
```

```
[3.31662479 3.46410162 3.60555128 3.74165739]
```

```
In [30]: # dot product
d1 = np.array([1,2,3])
dotproduct = np.dot(d1,m)
dotproduct
```

```
Out[30]: np.int64(74)
```

```
In [31]: print(d1)
print(m)
```

```
[1 2 3]
[11 12 13]
```

statistical function

```
In [32]: s = np.array([11,12,13,14])
mean = np.mean(s)
print(mean)
```

```
12.5
```

```
In [35]: # variance
var = np.var(s)
print(var)
```

```
1.25
```

```
In [36]: std = np.std(s)
print(std)
```

```
1.118033988749895
```

```
In [37]: min = np.min(s)
min
```

```
Out[37]: np.int64(11)
```

```
In [38]: max = np.max(s)
max
```

```
Out[38]: np.int64(14)
```

linear algebra

```
In [39]: mat = np.array([[1,2],[3,4]])
mat
```

```
Out[39]: array([[1, 2],
                [3, 4]])
```

Random Sampling Function

```
In [47]: random = np.random.rand(3)
print(random)
```

[0.891773 0.96366276 0.38344152]

```
In [45]: np.random.seed(0)
random = np.random.rand(4)
print(random)
```

[0.5488135 0.71518937 0.60276338 0.54488318]

```
In [60]: rand_int = np.random.randint(4)
print(rand_int)
```

3

```
In [64]: rand_int1 = np.random.randint(0,10,6)
print(rand_int1)
```

[7 3 2 7 2 0]

```
In [67]: np.random.seed(0)
rand_int2 = np.random.randint(0,10,6)
print(rand_int2)
```

[5 0 3 3 7 9]

```
In [72]: # Check if all elements are True
# all
logical_test1 = np.array([True, False, True])
all_true = np.all(logical_test1) # Check if all are True
print("All elements True:", all_true)
```

All elements True: False

```
In [70]: # Check if all elements are True
logical_test = np.array([False, False, False])
all_true = np.all(logical_test) # Check if all are True
print("All elements True:", all_true)
```

All elements True: False

```
In [71]: any_true = np.any(logical_test)
print(any_true)
```

False

```
In [73]: any_true = np.any(logical_test1)
print(any_true)
```

True

Set Operation

```
In [76]: set_a = np.array([1, 2, 3, 4])
set_b = np.array([3, 4, 5, 6])
```

```
intersection = np.intersect1d(set_a,set_b)
intersection
```

Out[76]: array([3, 4])

```
In [78]: union = np.union1d(set_a,set_b)
union
```

Out[78]: array([1, 2, 3, 4, 5, 6])

```
In [82]: # array attribute
a = np.array ([[1,2,3],[11,12,13]])
shape = a.shape
size = a.size
dimensions = a.ndim
dtype = a.dtype
print("Shape of a:", shape)
print("Size of a:", size)
print("Number of dimensions of a:", dimensions)
print("Data type of a:", dtype)
a
```

Shape of a: (2, 3)

Size of a: 6

Number of dimensions of a: 2

Data type of a: int64

Out[82]: array([[1, 2, 3],
[11, 12, 13]])

other functions

```
In [84]: a = np.array ([[1,2,3],[11,12,13]])
copied = np.copy(a)
copied
```

Out[84]: array([[1, 2, 3],
[11, 12, 13]])

```
In [86]: array_size_bytes = a.nbytes
array_size_bytes
```

Out[86]: 48

```
In [91]: # Check if two arrays share memory
shared = np.shares_memory(a, copied) # Check if arrays share memory
print("Do a and copied_array share memory?", shared)
print(id(a))
print(id(copied))
```

Do a and copied_array share memory? False

2613078464944

2613078471856

In []: